

NAME: Aaradhya Badola

SECTION: C

CU ID: CU240251570

ROLL NO.: 01

Experiment No. 5

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim:

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, and computer vision applications.

```
!pip install opencv-contrib-python scikit-image -q
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

print("All libraries imported successfully!")
```

All libraries imported successfully!

```
from google.colab import files

uploaded = files.upload()

print("Uploaded files:")
for filename in uploaded.keys():
    print(filename)
```

batman1.jpg
batman1.jpg(image/jpeg) - 116807 bytes, last modified: 9/15/2026 - 100% done
Saving batman1.jpg to batman1 (1).jpg
Uploaded files:
batman1 (1).jpg

```
import cv2

image1 = cv2.imread("batman.jpg")

image2 = cv2.resize(
    image1,
    None,
    fx=0.8,
    fy=0.8,
    interpolation=cv2.INTER_AREA
)

cv2.imwrite("batman_modified.jpg", image2)

print("Image 1: batman.jpg")
print("Image 2: batman_modified.jpg")
```

Image 1: batman.jpg
Image 2: batman_modified.jpg

```
import cv2
import matplotlib.pyplot as plt

# Load original and modified images
image1 = cv2.imread("batman.jpg")
image2 = cv2.imread("batman_modified.jpg")
```

```
# Check if images loaded
print("Original image loaded:", image1 is not None)
print("Modified image loaded:", image2 is not None)

# Convert BGR to RGB for displaying with Matplotlib
image1_rgb = cv2.cvtColor(image1, cv2.COLOR_BGR2RGB)
image2_rgb = cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)

# Display both images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image1_rgb)
plt.title("Original Batman Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(image2_rgb)
plt.title("Modified Batman Image")
plt.axis("off")

plt.show()
```

Original image loaded: True
Modified image loaded: True

Original Batman Image



Modified Batman Image



```
# Convert both images to grayscale
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Display grayscale images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap="gray")
plt.title("Grayscale - Original Batman")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap="gray")
plt.title("Grayscale - Modified Batman")
plt.axis("off")

plt.show()
```

Grayscale - Original Batman



Grayscale - Modified Batman



```
# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and calculate descriptors
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("SIFT Feature Extraction")
print("-----")
print("Keypoints in original image:", len(keypoints1))
print("Keypoints in modified image:", len(keypoints2))

print("Descriptor shape - original:", descriptors1.shape)
print("Descriptor shape - modified:", descriptors2.shape)
```

```
SIFT Feature Extraction
-----
Keypoints in original image: 1315
Keypoints in modified image: 870
Descriptor shape - original: (1315, 128)
Descriptor shape - modified: (870, 128)
```

```
# Draw SIFT keypoints
sift_result1 = cv2.drawKeypoints(
    gray1,
    keypoints1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

sift_result2 = cv2.drawKeypoints(
    gray2,
    keypoints2,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Display
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.imshow(sift_result1, cmap="gray")
plt.title("SIFT Keypoints - Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(sift_result2, cmap="gray")
plt.title("SIFT Keypoints - Modified")
plt.axis("off")

plt.show()
```

SIFT Keypoints - Original



SIFT Keypoints - Modified



✓ Observation

SIFT detected several keypoints in the Batman image. These keypoints are mainly located around distinctive edges, textures, facial features, and other important regions. SIFT keypoints can be used for image matching and object recognition.

```
from skimage.feature import hog

# Convert grayscale image to values between 0 and 1
gray1_float = gray1.astype("float32") / 255.0

# Extract HOG features
hog_features, hog_image = hog(
    gray1_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True
)

print("HOG Feature Extraction")
print("-----")
print("Number of HOG features:", len(hog_features))
print("Orientations:", 9)
print("Pixels per cell:", (8, 8))
print("Cells per block:", (2, 2))
```

```
HOG Feature Extraction
-----
Number of HOG features: 799236
Orientations: 9
Pixels per cell: (8, 8)
Cells per block: (2, 2)
```

```
from skimage import exposure

# Improve HOG visualization
hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap="gray")
plt.title("Original Batman Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
```

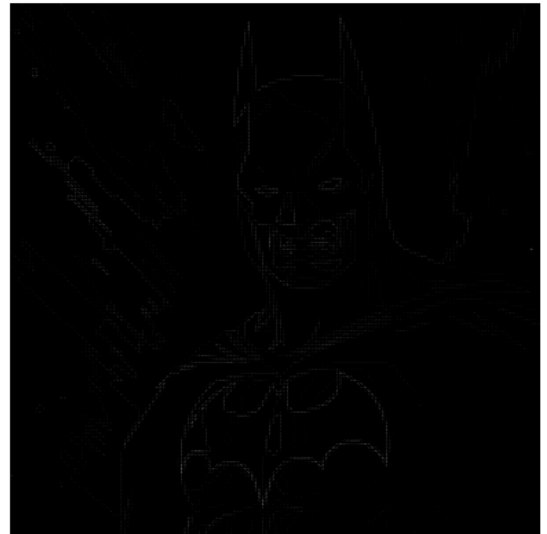
```
plt.title("HOG Visualization")
plt.axis("off")

plt.show()
```

Original Batman Image



HOG Visualization



Observation

HOG captures the gradient and edge information present in the Batman image. It represents the shape and structure of the object using the direction of image gradients.

✓ Comparison of SIFT and HOG

Property	SIFT	HOG
Feature type	Local feature descriptor	Gradient-based descriptor
Scale invariance	Yes	No
Rotation invariance	Yes	Limited
Main information	Keypoints and local features	Edges and shape
Computational complexity	Relatively higher	Relatively lower
Suitable for	Image matching and recognition	Object detection and shape recognition
Output	Keypoints and descriptors	Feature vector

SIFT is more suitable for matching distinctive features between images. HOG is more suitable for describing the shape and edge structure of objects.

```
# Create Brute Force Matcher
bf = cv2.BFMatcher()

# Find the two best matches for each descriptor
matches = bf.knnMatch(descriptors1, descriptors2, k=2)

# Apply Lowe's ratio test
good_matches = []

for pair in matches:
    if len(pair) == 2:
        m, n = pair

        if m.distance < 0.75 * n.distance:
            good_matches.append(m)

print("Total matches:", len(matches))
print("Good matches:", len(good_matches))
```

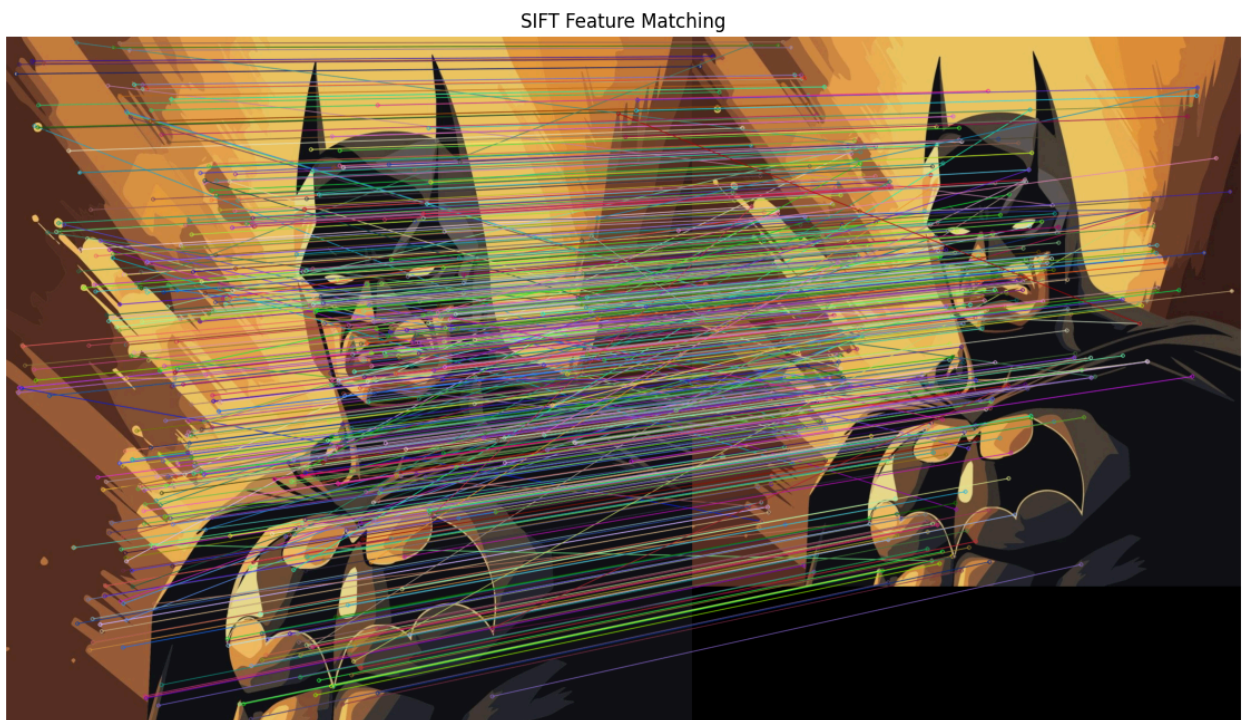
```
Total matches: 1315
Good matches: 628
```

```
# Draw good matches
matched_image = cv2.drawMatches(
```

```
image1,
keypoints1,
image2,
keypoints2,
good_matches,
None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Convert for Matplotlib
matched_image_rgb = cv2.cvtColor(
    matched_image,
    cv2.COLOR_BGR2RGB
)

# Display
plt.figure(figsize=(16, 8))
plt.imshow(matched_image_rgb)
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()
```



Observation

SIFT successfully matched several local features between the original Batman image and its modified version. The matching lines indicate similar feature points detected in both images. This demonstrates the usefulness of SIFT for image matching.

Strengths and Limitations

SIFT

Advantages:

- Scale invariant.
- Rotation invariant.
- Good for image matching.
- Useful for object recognition.
- Detects distinctive local features.

Limitations:

- Relatively computationally expensive.

- Can generate many features.
- May be less effective in areas with very little texture.

HOG

Advantages:

- Good for representing object shape.
- Captures edge and gradient information.
- Useful for object detection.
- Relatively simple to implement.

Limitations:

- Not scale invariant.
- Limited rotation invariance.
- Can be affected by significant changes in viewpoint.

Overall Observations and Applications

Observations

1. SIFT successfully detected distinctive keypoints from the Batman image.
2. The keypoints were mainly found around important edges and textured regions.
3. HOG captured the gradient and edge structure of the image.
4. SIFT descriptors were used to match features between the original and modified Batman images.
5. SIFT is more suitable for feature matching because it is scale and rotation invariant.
6. HOG is useful for representing object shape and edge information.
7. Both techniques convert raw image information into useful feature representations.

Applications of SIFT

- Image matching
- Object recognition
- Image stitching
- Object tracking

Applications of HOG

- Pedestrian detection
- Human detection
- Object detection
- Shape recognition

Conclusion

In this experiment, SIFT and HOG feature extraction techniques were successfully implemented using Python, OpenCV, and scikit-image. SIFT was used to detect keypoints and generate local feature descriptors, while HOG was used to represent gradient and shape information. SIFT demonstrated its usefulness for image matching, while HOG was useful for representing object shape and edge information.

✓ Questions and Answers

1. What is feature extraction, and why is it important in computer vision?

Feature extraction is the process of obtaining important and meaningful information from an image. Instead of using every pixel of an image, feature extraction represents the image using useful features such as edges, corners, textures, shapes, and patterns.

It is important because it reduces the amount of data and makes it easier for computer vision systems to recognize, classify, compare, and detect objects in images.

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

SIFT is a feature extraction technique used to detect distinctive points in an image and create descriptors for those points.

The basic working of SIFT is:

1. It searches for important points called keypoints at different scales of the image.
2. It identifies stable keypoints using differences between blurred images.
3. It assigns an orientation to each keypoint.
4. It creates a descriptor containing information about the local image region around the keypoint.
5. These descriptors can be compared between images to find matching features.

SIFT is useful because its features remain relatively stable when the image is resized or rotated.

3. What are keypoints and feature descriptors in image analysis?

Keypoints are important or distinctive locations in an image. Examples include corners, edges, or areas with significant changes in intensity.

Feature descriptors are numerical representations of the area surrounding a keypoint. They contain information that helps identify and compare the same feature in another image.

For example, in our experiment, SIFT detects keypoints in the Batman image and generates descriptors for those keypoints.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

HOG is a feature descriptor that represents the shape and appearance of an object using the directions of image gradients.

The image is divided into small cells. For each cell, the directions and strengths of edges are calculated and represented as a histogram. These histograms are combined to form the HOG feature vector.

HOG is significant because it is effective at representing the shape and edge structure of objects. It is commonly used in object detection systems.

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Property	SIFT	HOG
Main feature	Local keypoints	Edges and gradients
Robustness	Strong against scale and rotation changes	Works well for shape and edge information
Computational complexity	Relatively high	Generally lower
Scale invariance	Yes	No
Rotation invariance	Yes	Limited
Applications	Image matching, object recognition	Object detection, pedestrian detection

SIFT is generally better for matching features between images, while HOG is useful when the shape and edge structure of an object are important.

6. Why is SIFT considered invariant to scale and rotation?

SIFT is considered scale invariant because it searches for keypoints at multiple scales. Therefore, important features can still be detected when an object appears larger or smaller.

It is considered rotation invariant because SIFT assigns an orientation to each keypoint based on the local image gradients. The descriptor is then created relative to this orientation.

As a result, the same feature can be recognized even when the image has been rotated or resized.

7. Mention three real-world applications where HOG descriptors are commonly used.

Three common applications of HOG are:

1. **Pedestrian detection** — detecting people in images or video.
2. **Human detection** — identifying human shapes in surveillance and computer vision systems.
3. **Object detection** — detecting objects based on their shape and edge information.

8. Why is feature extraction performed before image classification or object detection?

Feature extraction is performed to convert raw image data into meaningful information that a computer vision algorithm can process more easily.

An image may contain thousands or millions of pixels. Processing all these pixels directly can be computationally expensive. Features such as edges, shapes, textures, and keypoints provide a more useful representation of the image.

Therefore, feature extraction can make classification and object detection faster and more effective.

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Advantages of handcrafted descriptors:

- They are relatively simple to understand and implement.
- They can work with smaller datasets.
- They generally require less computational power than large deep learning models.
- Their features have a clear meaning, such as edges, gradients, or keypoints.

Limitations:

- Their performance depends heavily on manually selected features and parameters.
- They may not work well with large variations in objects, lighting, and viewpoints.
- They are generally less flexible than deep learning methods for complex recognition tasks.

Deep learning methods can automatically learn complex features from large datasets, but they usually require more data and computational resources.

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Feature extraction provides useful information that allows computer vision systems to compare and identify images and objects.

Image matching: Features from two images can be compared to find similar regions. SIFT descriptors are commonly used for this purpose.

Face recognition: Features representing important facial structures can be extracted and compared with stored features to identify a person.

Object detection: Features such as edges, shapes, textures, and gradients help a system identify objects in an image. HOG, for example, can represent the shape of objects such as humans.

Therefore, feature extraction acts as an important intermediate step between the raw image and the final recognition or detection result.

Double-click (or enter) to edit